The Architecture of Claude Code Skills

Most people who write a Claude Code skill create a markdown file, add a description, and wonder why Claude ignores it. The skill triggers sometimes, does the right thing occasionally, and feels more like a coin flip than an engineering artifact. The frustration is real, and it points to something deeper than bad luck: a misunderstanding of what skills are and how they work.

Skills are not documents that Claude reads. They are not configuration files. They are not prompts in fancy packaging. They are agent toolkits — folders containing instructions, executable code, reference material, and configuration that Claude explores, uses, and returns to. The architecture governing how these toolkits load, trigger, and interact with the rest of the system is not arbitrary. It emerges from a single economic constraint — the description budget — and the engineering response to that constraint is a pattern called progressive disclosure. Understanding both is the difference between a skill that works reliably and one that sits unused.

A Skill Is Not a Document

A skill is a folder that Claude explores, not a file it reads. The SKILL.md is the entrypoint, but the directory around it matters just as much. A well-built skill folder might contain a `scripts/` directory with validation tools and language detection, a `references/` directory with domain-specific documentation, and an `assets/` directory with templates or configuration files. Claude discovers these through directory exploration the same way a developer navigates a codebase — listing what's there, reading what's relevant, ignoring what isn't.

Anthropic organizes their hundreds of internal skills into nine purpose categories. Library and API reference skills teach Claude about internal tools. Product verification skills pair with testing frameworks to confirm code works. Data fetching skills connect to monitoring stacks with pre-built credential handling. Business process

skills automate repetitive workflows and keep logs of previous runs for consistency. Code scaffolding skills generate boilerplate where natural language requirements go beyond what templates alone can express. Code quality skills enforce standards, sometimes spawning adversarial subagents that iterate until findings degrade to nitpicks. CI/CD skills babysit pull requests, retry flaky tests, and resolve merge conflicts. Runbooks walk through multi-tool investigations starting from a symptom. Infrastructure operations skills find orphaned resources, post findings, wait for confirmation, and execute cascading cleanup. The best skills fit one category cleanly. The confusing ones straddle several.

The most powerful thing you can put in a skill folder is code. Giving Claude scripts means it spends its reasoning turns on composition — deciding what to do next and how pieces fit together — rather than reconstructing the same boilerplate it wrote in the previous session. A data-science skill with helper functions for fetching from an event source lets Claude generate analysis scripts on the fly instead of rewriting HTTP plumbing every time someone asks what happened on Tuesday. When Anthropic's test harness reveals that three independent test runs all produced the same helper script, that is a clear signal: bundle it. Write it once, put it in `scripts/`, and save every future invocation from reinventing the wheel.

Every Skill Pays a Tax

The description budget is the scarce resource that governs every skill design decision. Claude scans a list of skill names and descriptions to decide whether any skill is relevant to the current request. All descriptions share a budget calculated as two percent of the context window, with a fallback of sixteen thousand characters. Each skill consumes its description length plus about a hundred characters of structural overhead. At typical description lengths, this means thirty to forty skills fit comfortably. Go beyond that, and skills start crowding each other out. Truncation is silent — skills simply vanish from Claude's awareness behind a hidden HTML comment. In one documented case with sixty-three installed skills, twenty-one were completely invisible to the model.

This budget exists because of how prompt caching works. Claude Code structures its prompt into layers optimized for cache reuse: a static system prompt

cached globally across all users, project-specific instructions cached per project, session context cached per session, and conversation messages rebuilt per turn. Skill descriptions live in the cached prefix, the part of the prompt that stays stable across requests. Anthropic monitors cache hit rate the way an operations team monitors uptime — drops trigger alerts. Adding or removing tools mid-session invalidates the entire cache. Switching models mid-session forces a cache rebuild that costs more than staying on the original model. The engineering team has built the system around prefix stability, and skill descriptions are part of that prefix.

The practical consequence is that every word in a description is paid for on every request, whether the skill is relevant or not. Fifteen programming subskills at a hundred words each consume forty percent of the total budget even when the user is writing documentation or analyzing data. This is not a theoretical concern. It is the central design constraint, and the architecture exists to manage it.

Three Levels of Progressive Disclosure

Skills load in three stages, each with dramatically different costs. The first level — the skill's name and description — is always present, costing about a hundred tokens per skill. This is the tax. The second level — the SKILL.md body — loads only when Claude decides the skill is relevant and triggers it, consuming roughly one to two percent of the available context. The third level — reference files, scripts, and assets — loads only when Claude explicitly reads them during the course of working on the task.

The file system is the progressive disclosure mechanism. Claude navigates a skill folder the way a developer navigates an unfamiliar project. It lists directories, reads files it needs, and ignores files it doesn't. The SKILL.md body serves as an orientation guide, pointing Claude toward what exists — which reference files cover which topics, which scripts handle which tasks, what assets are available. Without these pointers, Claude doesn't know the references exist. With them, Claude loads only the reference files relevant to the current task, keeping the rest out of its context window entirely.

Scripts run without consuming context at all. This is one of the most underappreciated capabilities of the skill architecture. A validation script, a language-detection script, a test runner — these execute and return their results without their source code ever occupying space in the conversation. The skill preserves context for

reasoning and composition while delegating deterministic work to code that runs outside the context window. This is fundamentally different from a reference file, which must be read into context to be useful.

The Description Is a Trigger, Not a Summary

Claude decides whether to consult a skill by reasoning about the user's request against the set of available skill descriptions. This is not keyword matching — it is semantic reasoning about whether the skill would be helpful. But there is a crucial asymmetry: Claude only consults skills for tasks it cannot handle easily on its own. Simple, one-step queries rarely trigger skills even when the description matches perfectly, because Claude can handle them without specialized guidance. Testing a skill's triggering against trivial prompts produces misleadingly low activation rates.

The most common failure mode is undertriggering. Claude errs toward not using skills, which means the default state of a new skill is neglect. Anthropic's internal guidance is to make descriptions aggressive. The instruction is literally to make them "pushy" — to include specific phrases users would actually say, specific error messages they might encounter, specific contexts and symptoms that should activate the skill. Anthropic's own skill descriptions include language like "even if they don't explicitly ask for a dashboard" to push through Claude's natural reluctance. The difference between a vague description and a well-crafted one routinely moves activation rates from twenty percent to ninety.

A description that summarizes the workflow becomes the workflow, and that is a bug. One team discovered that including "dispatches subagent per task with code review between tasks" in a description caused Claude to perform only one code review even though the full skill body specified two separate reviews. Claude treated the description as sufficient instruction and never read deeply enough into the body to discover the second review. The fix was to strip all workflow details from the description and replace them with pure triggering conditions. The description says when to activate. The body says what to do.

Descriptions can be optimized empirically with a train-test methodology. Anthropic's approach generates twenty test queries — half that should trigger the skill,

half that represent near-misses that shouldn't — and splits them into training and held-out test sets. The system evaluates the current description three times per query to get a reliable trigger rate, proposes improvements using extended thinking based on what failed, re-evaluates each candidate on both sets, and selects the winner by test-set accuracy to prevent overfitting. The near-miss queries are the most valuable: prompts that share keywords with the skill but actually need something else. A negative test case like "write a Fibonacci function" teaches nothing about a PDF skill's description quality. A negative case like "extract the text from this screenshot and save it as a document" — which sounds similar to PDF processing but is actually OCR — reveals whether the description has real discriminating power.

What Makes Skill Content Effective

Gotchas are the highest-signal content in any skill. Anthropic's assessment is unequivocal: if a skill has no gotchas section, it has not been used enough. Gotchas are built from observed failure points — the specific, concrete ways Claude goes wrong when attempting the task without guidance. A gotcha is not a generic best practice. It is a particular mistake that was actually observed, described precisely enough that Claude can recognize when it is about to make the same mistake again. Gotchas should be the first section added to a new skill and the primary focus of every maintenance pass.

The most effective instructions explain why, not just what. Anthropic's internal guidance warns against what they call "heavy-handed musty MUSTs." When a skill author writes ALWAYS or NEVER in capital letters, that is a signal to stop and ask whether the reasoning can be explained instead. Claude has theory of mind. When it understands why a practice matters — the incident that motivated the rule, the failure mode it prevents, the stakeholder it protects — it handles edge cases that rigid constraints would miss. A rule that says "NEVER mock the database in these tests" is less effective than an explanation that last quarter, mocked tests passed while a production migration broke because the mocks diverged from the actual schema. The explanation carries the rule AND the judgment about when it applies.

Effective skill content pushes Claude out of its defaults, not back into them. Anthropic built the frontend-design skill through iterative customer feedback specifically to override Claude's tendency to reach for the same font and the same

purple gradient palette. Restating what Claude already knows wastes context. Teaching the non-obvious — the company-specific conventions, the counter-intuitive best practices, the domain knowledge that only comes from experience with this particular codebase — is what makes a skill worth its budget cost.

Different skill types demand different structural patterns. Discipline skills — those enforcing practices like test-driven development or verification protocols — need anti-rationalization mechanisms. These include tables that map common excuses to reality checks, lists of red-flag thoughts that mean "stop and reconsider," and explicit closures of specific loopholes the model might exploit. Technique skills need templates, reference tables, and step-by-step procedures that provide structure without removing judgment. Workflow skills need cross-references and integration points with other skills. Reference skills need comparison tables and architectural context. Choosing the wrong structure for the skill type produces content that is technically correct but practically useless.

Organizing Complex Skill Systems

When a domain has many subspecialties, separate skills consume the description budget rapidly. A programming skill chain with subspecialties for Swift, TypeScript, C++, PostgreSQL, SwiftUI, concurrency, and several workflow phases can easily require fifteen separate descriptions. At a hundred words each, that is fifteen hundred words — forty percent of the total budget — dedicated to a single domain that the user may not need in a given session. Meanwhile, every other skill on the system is fighting for the remaining sixty percent.

The routing pattern consolidates subspecialties under a single description. Instead of fifteen skills each paying the description tax, create one skill with a single description that covers the full triggering surface. The SKILL.md body acts as a router, directing Claude to the appropriate reference file based on what it detects in the project. Swift-specific guidance lives in `references/swift.md` and loads only when Claude encounters Swift code. TypeScript patterns live in `references/typescript.md`. Workflow phases — implementation, dispatch, auditing, completion verification — each have their own reference file loaded at the appropriate moment. Fourteen descriptions

are eliminated, returning roughly seven thousand characters to the budget. The content is preserved; only the loading mechanism changes.

Skills can register component-scoped hooks via frontmatter. A skill's YAML frontmatter can include a `hooks` key that defines event handlers registered when the skill activates and cleaned up when it finishes. A programming skill might register a PreToolUse hook that validates Bash commands against a safety script, active only while the skill is running. All twenty-three hook event types are supported, with four handler types: shell commands for deterministic checks, HTTP endpoints for external integration, single-turn LLM prompts for context-aware validation, and multi-turn subagents for complex verification. These hooks add behavior — validation, guardrails, measurement — not content. They complement the routing pattern rather than replacing it.

For hooks that need to outlive the skill, flag-file patterns provide session-scoped persistence. A skill can create a flag file when it activates, and pre-existing hooks defined in settings.json can check for that flag before executing their logic. A programming-specific validation hook might check for `.programming-active` in the project directory: if the file exists, run the validation; if not, exit silently. Settings-level hooks load at session start and cannot be added or removed mid-session, but their behavior can be conditional on state that the skill controls.

Skills Evolve Through Use

A skill without observed failures is a skill that has not been tested seriously. The right lifecycle begins with a draft, runs it against realistic scenarios, observes where Claude goes wrong, adds gotchas and corrections, and tests again. Anthropic's own AskUserQuestion tool took three complete redesigns before it worked. The first two versions were technically sound but Claude did not gravitate toward using them. The third worked because it matched the model's natural inclination — it felt right to call. Skills that are written once and never revised are leaving performance on the table.

Anthropic tests skills by running them head-to-head against baselines. For each test prompt, two subagents run in parallel: one with the skill loaded, one without. Both produce outputs that are graded against assertions — programmatic checks where possible, human judgment through a browser-based review interface where not — and

aggregated into benchmarks showing pass rate, time, and token usage with statistical variance. The comparison reveals whether the skill actually helps. Sometimes the baseline performs equally well, which means the skill is adding context overhead without improving outcomes. Sometimes the skill actively hurts performance by making the model follow instructions that are worse than its default behavior. The only way to know is to measure.

Skills need revisiting as models improve. Structures and guardrails that helped weaker models may actively constrain stronger ones. The transition from TodoWrite to the Task system illustrates this precisely: reminder mechanisms that helped earlier models stay on track caused improved models to think they could not modify the task list, treating it as immutable rather than as a living plan. Every skill contains assumptions about the model's capabilities. When those capabilities change — when the model gets better at planning, or better at multi-file edits, or better at self-correction — instructions that compensated for earlier weaknesses become unnecessary weight. Periodic review with fresh eyes and a willingness to cut is as important as the initial writing.

The architecture of Claude Code skills is, at bottom, a resource allocation problem. A fixed budget of sixteen thousand characters governs what Claude sees on every request. Progressive disclosure — the three-level loading system from description to body to references — is the mechanism for making that budget stretch across dozens of skills without drowning every session in irrelevant context. The craft of skill authoring sits at the intersection of these constraints: writing descriptions that trigger reliably without wasting characters, building content that pushes Claude past its defaults without restating what it already knows, organizing complex domains so they pay a single description tax instead of fifteen, and revisiting everything as the model beneath it improves. The skill is not the markdown file. The skill is the entire folder, evolving through observed failures, earning its place in the budget by making Claude measurably better at the task.